DeepEP v2

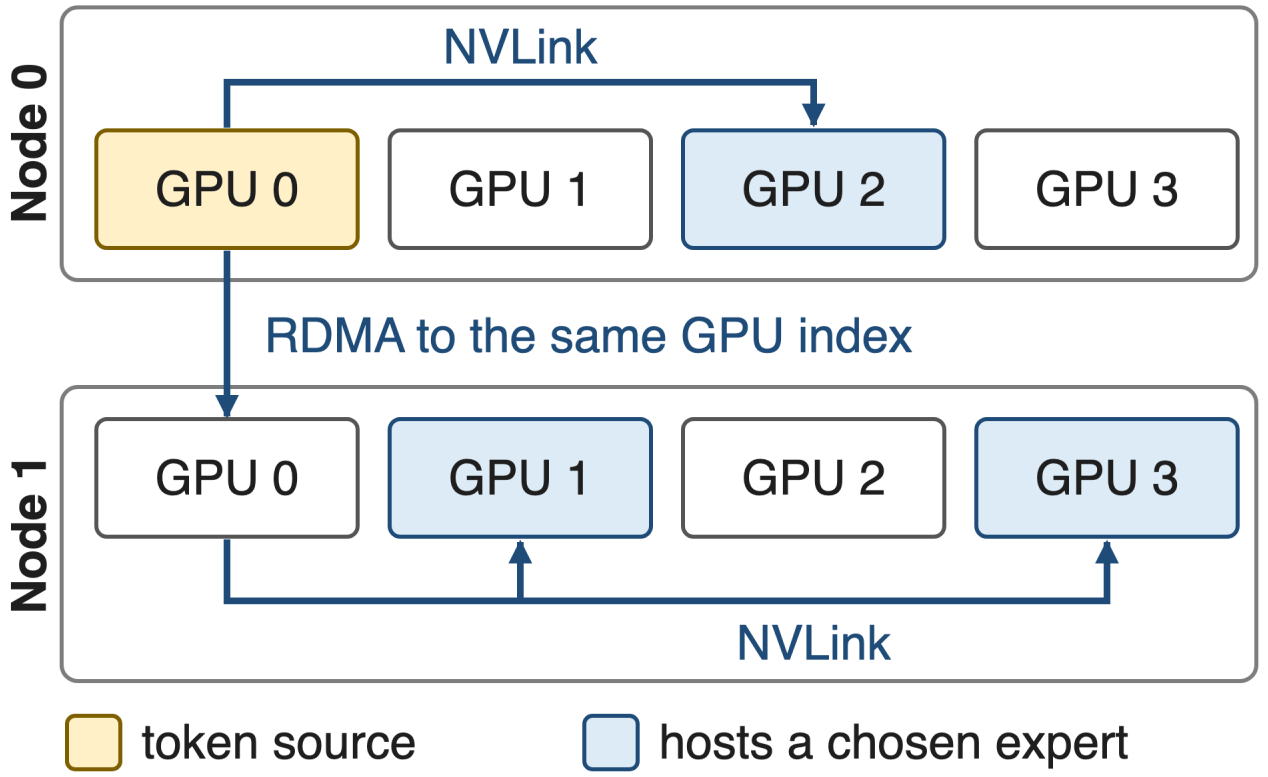
DeepSeek's expert-parallel communication library, rebuilt on NCCL Gin

DeepSeek-AI · DeepEP V2 (Apr 2026) and V2.5 (Sep 2026)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Tokens cross two networks twice per MoE layer

- DeepSeek-V3: 256 routed experts, top-8, at most 4 nodes per token
- NVLink 160 GB/s inside a node, InfiniBand 50 GB/s across
- Combine retraces the path in reverse



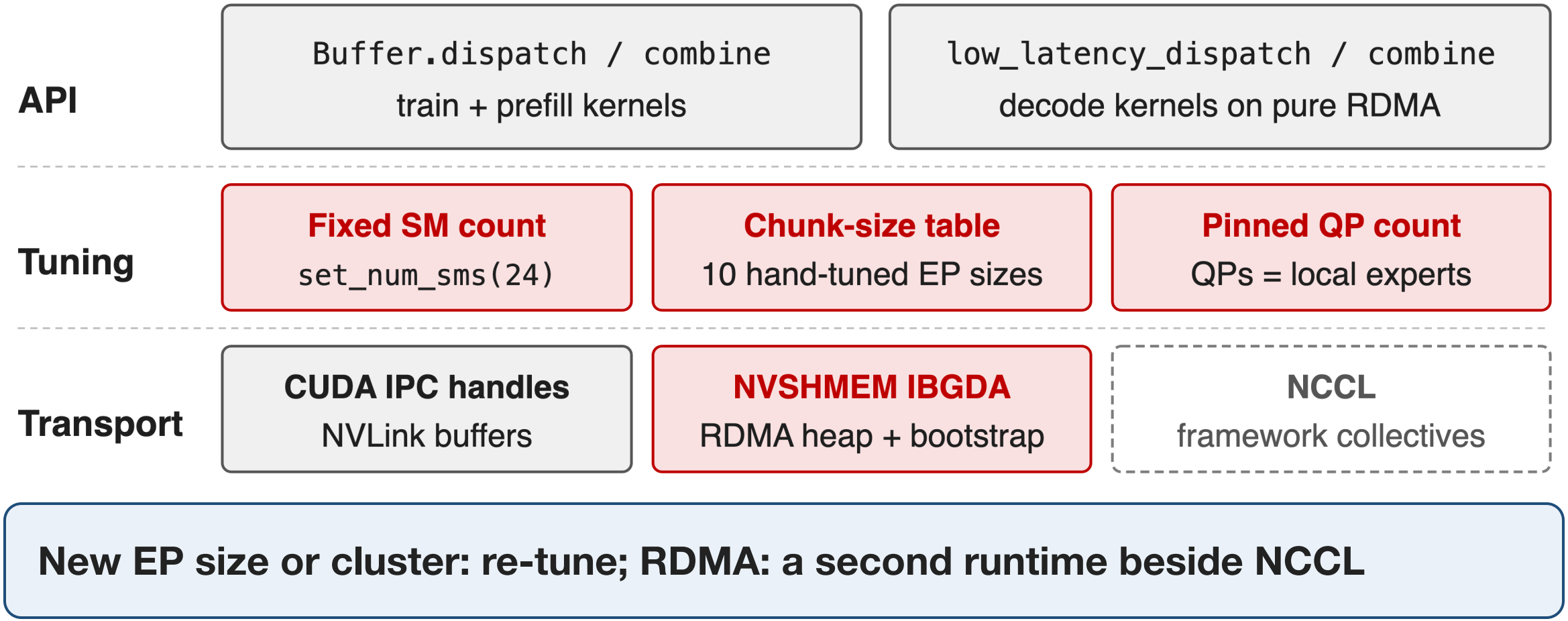
All-to-all takes 15% of every GPU's SMs

20 of 132

H800 SMs DeepSeek-V3 reserves for all-to-all, at a 1:1 compute-to-communication ratio

DeepSeek-V3 Technical Report (arXiv 2412.19437) §3.5.1 and §3.2.1; 15% = 20/132 (derived)

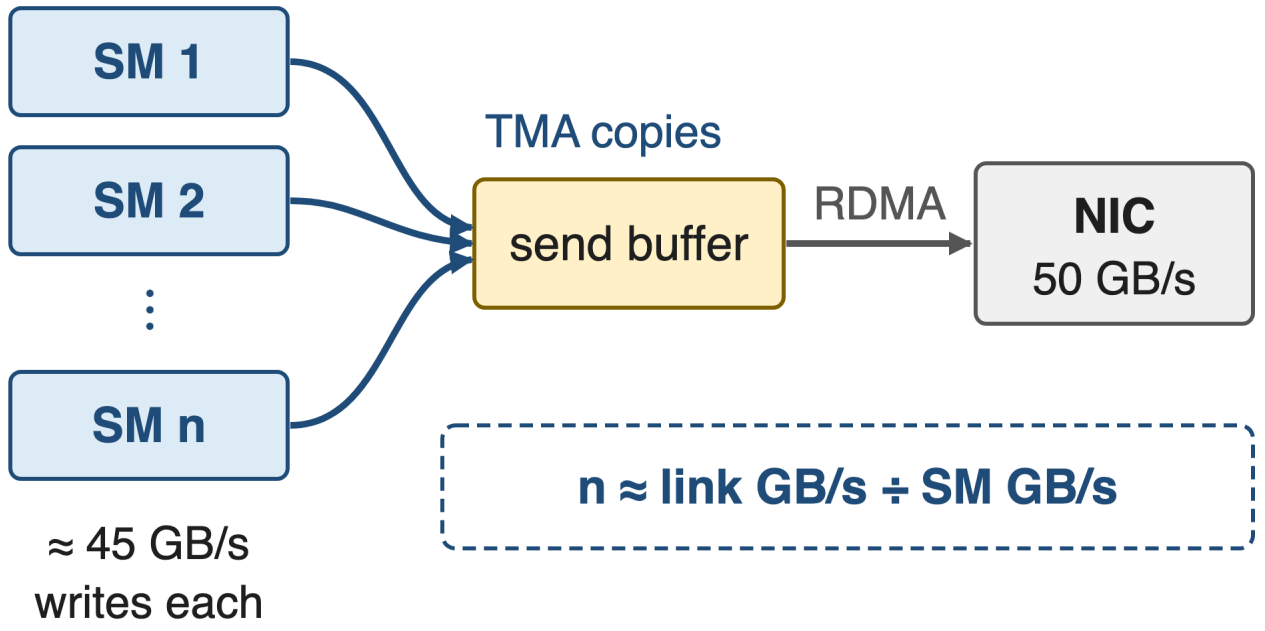
V1 grew two kernel families and a tuning table



New EP size or cluster: re-tune; RDMA: a second runtime beside NCCL

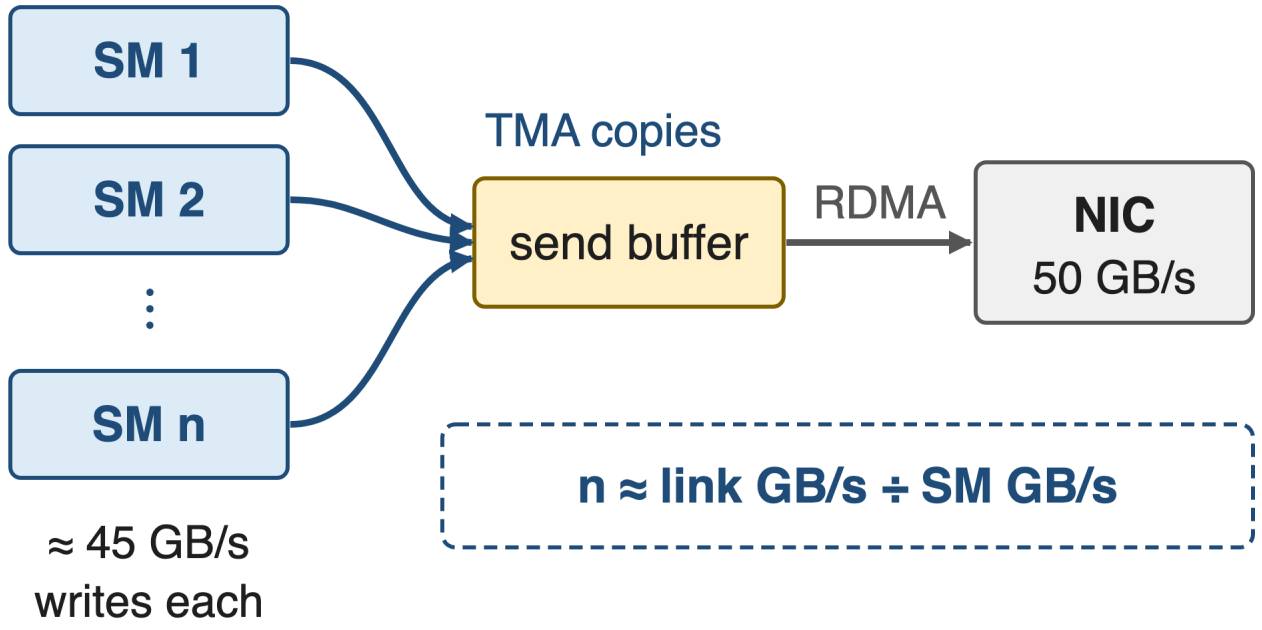
Insight: SMs only need to feed the slowest link

- One SM writes ≈ 45–50 GB/s (model constant)
- A 400 Gb/s NIC drains 50 GB/s
- Routing statistics fix the bytes per token



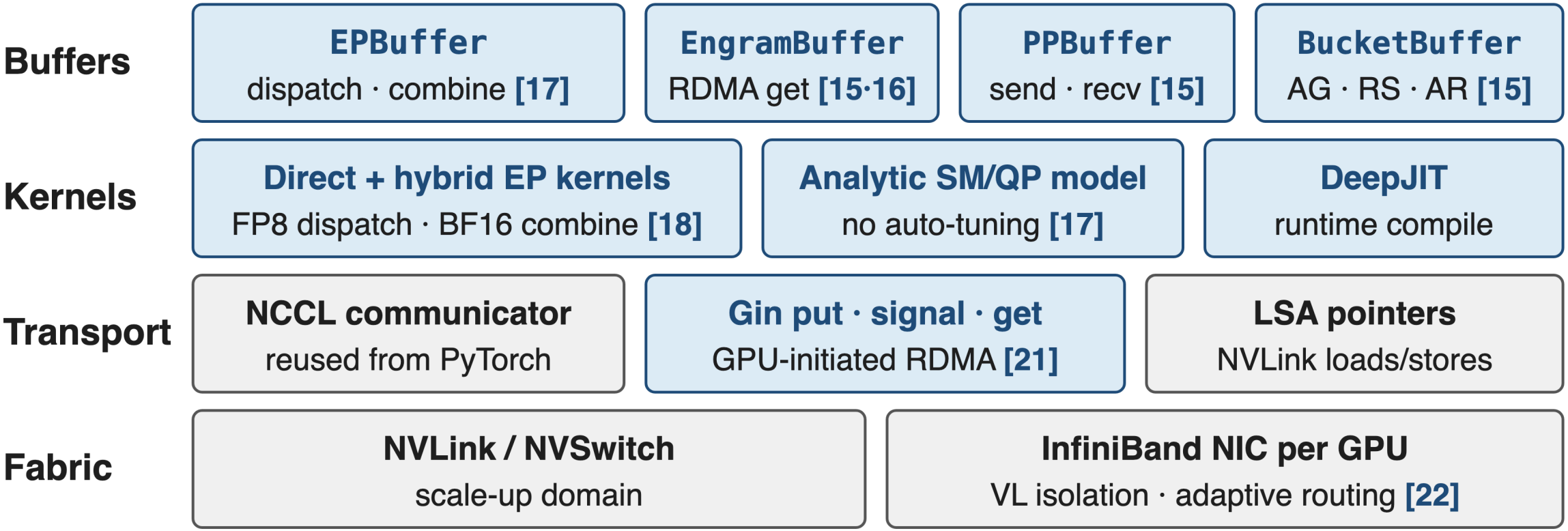
Insight: SMs only need to feed the slowest link

- One SM writes ≈ 45–50 GB/s (model constant)
- A 400 Gb/s NIC drains 50 GB/s
- Routing statistics fix the bytes per token



Compute the SM/QP budget; let the NIC move the bytes

Four layers share one NCCL communicator



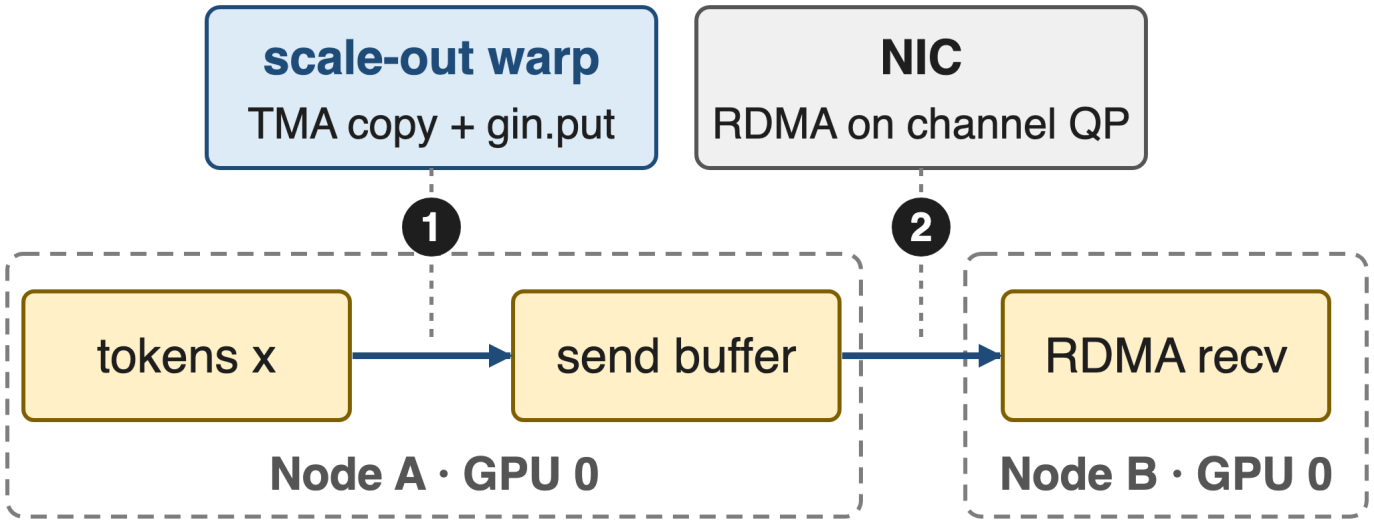
Accent = DeepEP-owned, gray = NCCL / hardware, [N] = sibling explainer deck. V2.0 had one ElasticBuffer; V2.5 split it.

❶ One EPBuffer serves training, prefill and decode

- `dispatch` returns expert-major rows plus a `handle`; `combine(y, handle)` reverses it
- Train, prefill: CPU sync for exact sizes; decode: GPU-side counts, CUDA-graph friendly
- Rows aligned for grouped GEMM; backward replays the cached handle

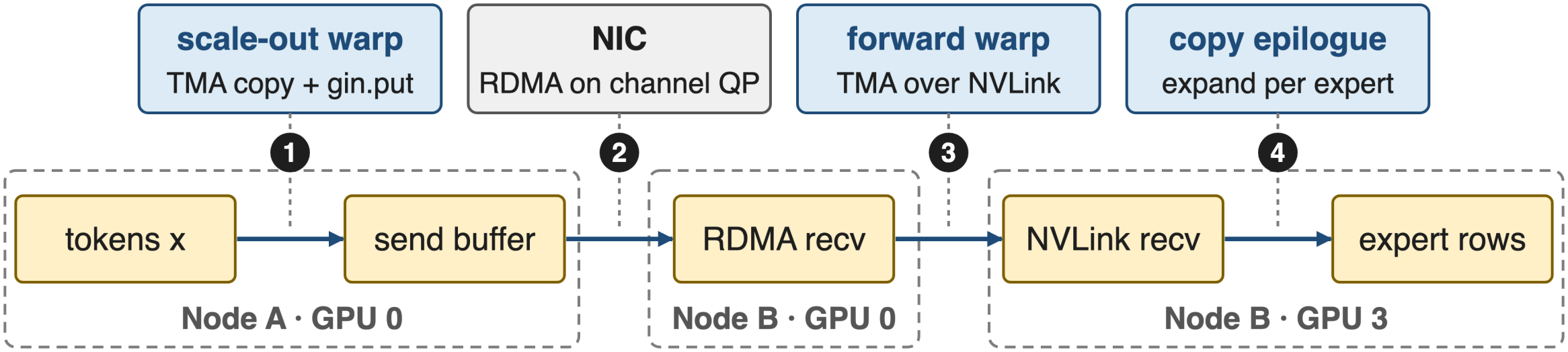
Deep dive: deck 17. Decode-side GEMM over fixed capacity: deck 34

❷ Hybrid dispatch: rail RDMA, then NVLink



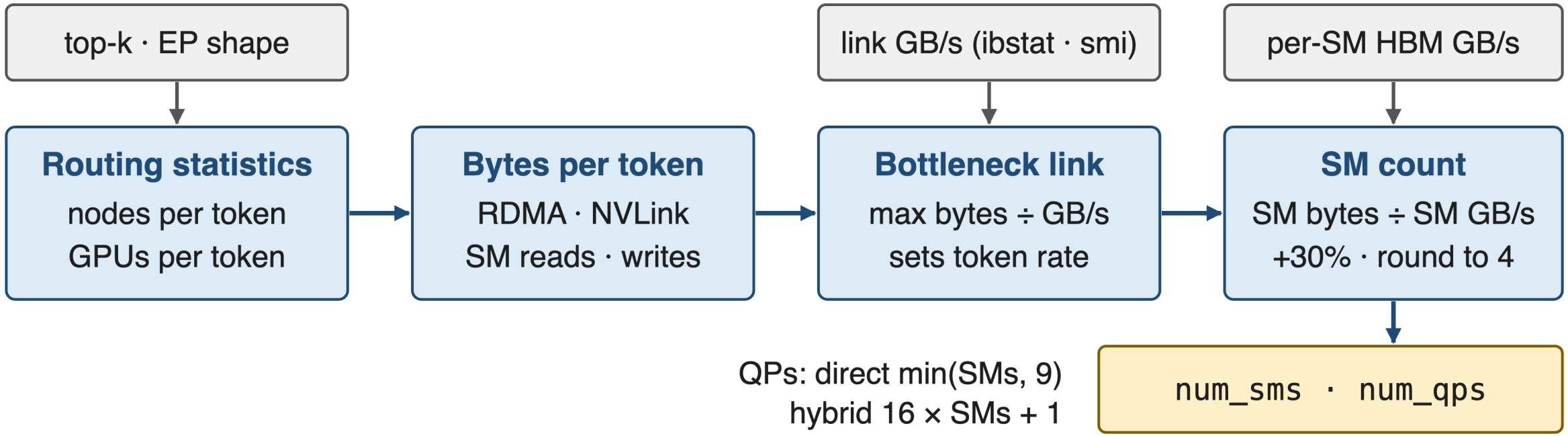
Each warp is a channel with its own QP when enough are allocated; 4 notify warps per SM first exchange per-rank and per-expert counts

❷ Hybrid dispatch: rail RDMA, then NVLink



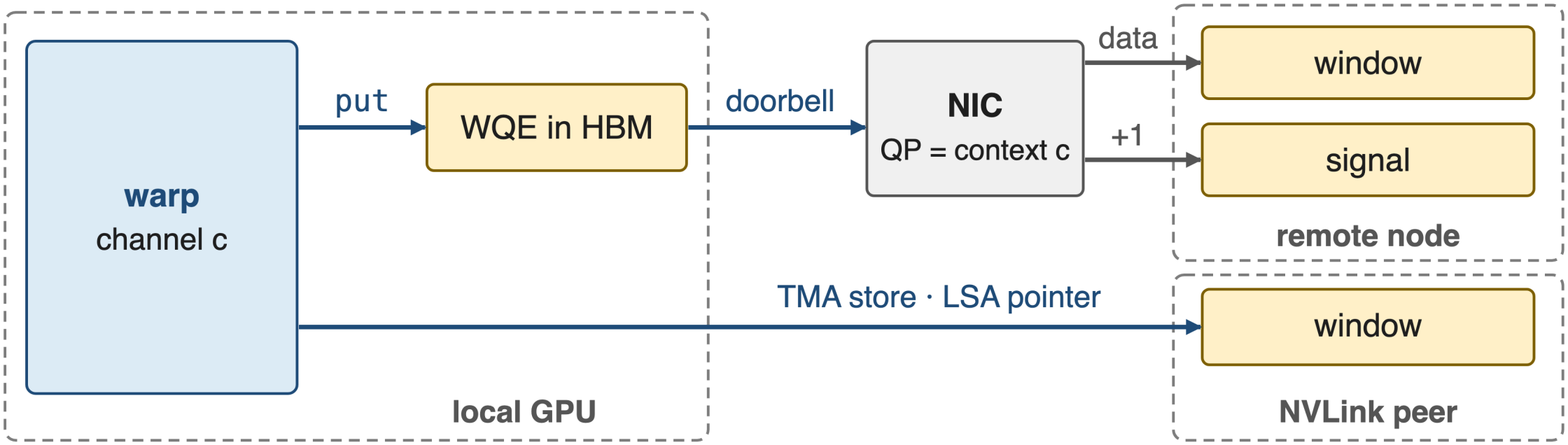
Forward warps poll Gin tail signals, then TMA-store over NVLink; with one domain, direct mode writes straight to the destination

❸ A formula sets SM and QP counts



Computed once per buffer and cached; a worked EP 8×4 example is in deck 17

❹ Warps post RDMA writes through NCCL Gin



One symmetric window registered on PyTorch's own NCCL communicator; V2.5 removed NVSHMEM entirely

The signal lands after the data on the same QP. Deep dive: deck 21

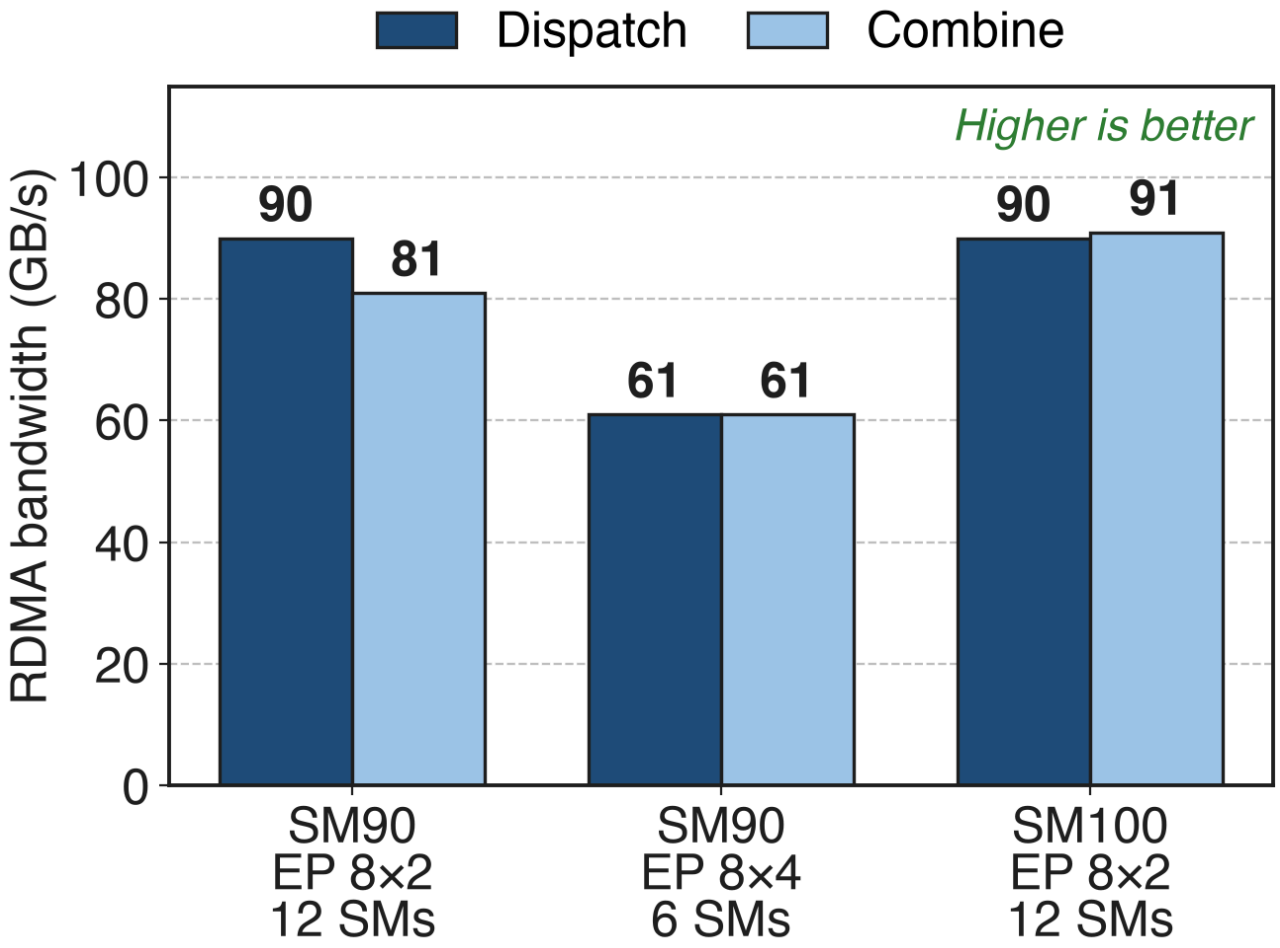
Gin also carries PP, Engram and collectives

- `PPBuffer`: RDMA send and receive between pipeline neighbours (deck 15)
- `EngramBuffer`: RDMA gets from GPU or CPU tables (decks 15, 16)
- `BucketBuffer`: all-gather on copy engines, reduce-scatter on SMs (deck 15)

V2.5 also adds weight prefetch and gradient reduce for redundant experts

6–12 SMs drive 61–91 GB/s of RDMA per GPU

- V3 setup: 8K tokens, hidden 7168, top-8
- FP8 dispatch, BF16 combine, CX7 NICs
- Logical bandwidth, local-rank traffic included



V3-style training drops from 24 SMs to 4–6

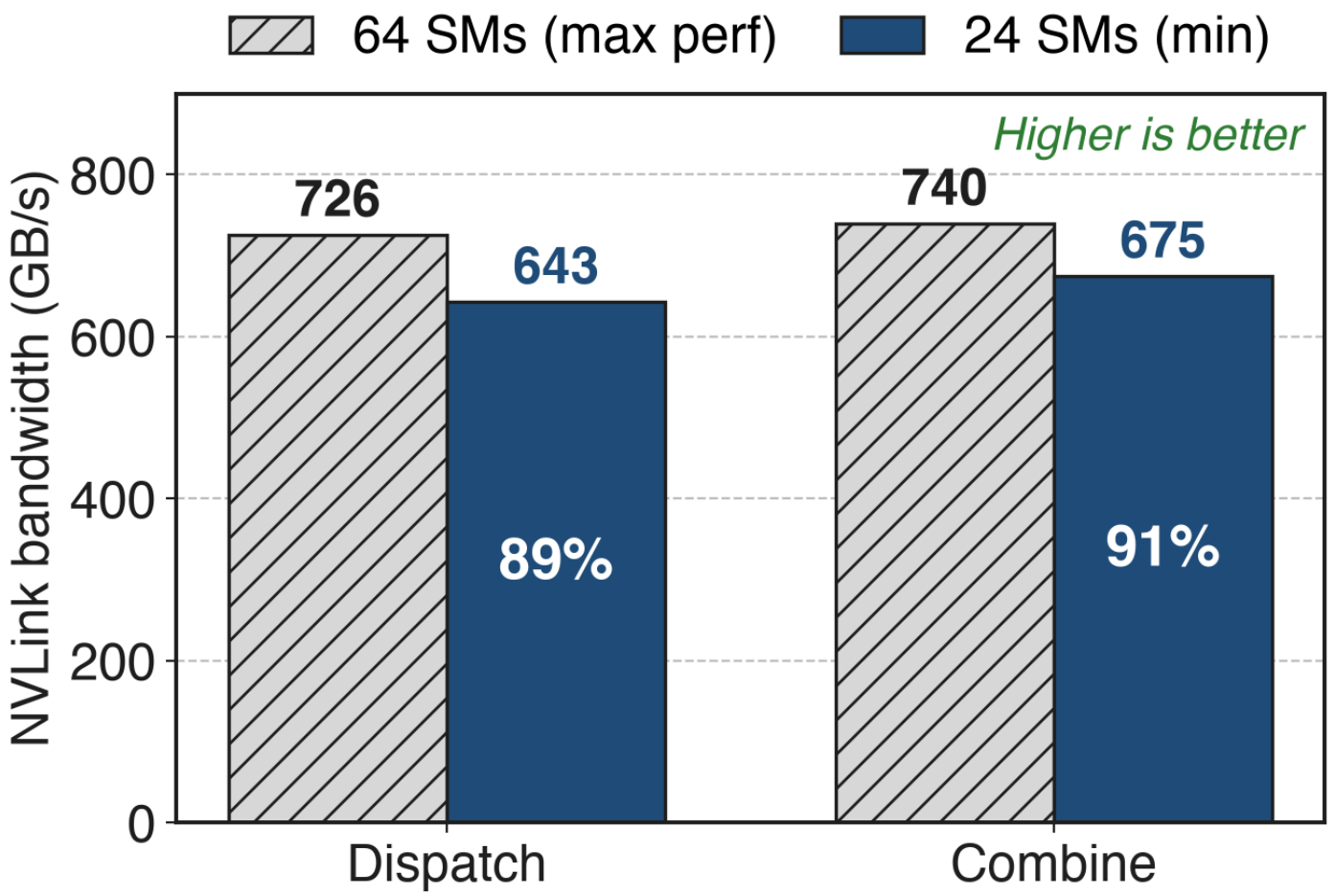
24 → 4–6

communication SMs per GPU for V3-like training, at equivalent or better speed; vs V1 up to 1.3× peak with up to 4× fewer SMs

DeepEP V2 release notes, README at commit b306af0 and PR #605

24 SMs keep 89–91% of peak NVLink bandwidth

- SM100, EP8 inside one NVLink domain
- 64 SMs: the max-performance setting
- 24 SMs: 2.7× fewer SMs (derived)



Adopted widely, but needs bigger buffers

- Megatron-LM FlexDispatcher: `deepepv2` backend
- DeepEP-Ascend: same API on Ascend 950
- Ports: UCCL-EP, Mori-EP for AMD

- **Larger buffers** than V1
- **Hopper+, CUDA 13.1, NCCL 2.32.3**
- EP needs SMs; Engram, PP, buckets experimental

Summary

- DeepSeek-V3 all-to-all: **20 of 132** SMs per GPU
- **Insight**: SMs only feed the slowest link; compute the budget
- One EPBuffer, analytic SM/QP, NCCL Gin transport
- **24 → 4–6** SMs; **61–91 GB/s** RDMA from 6–12 SMs
- Decks 15–18, 21, 22; related 19, 34

https://github.com/deepseek-ai/DeepEP